

Documentazione tecnica

Sistema VLM + YOLO + AI2-THOR

Pseudocodice, architettura e flusso completo

Data: 03/02/2026

Questo documento descrive in modo completo come parte il sistema, come si collegano AI2-THOR, YOLO e VLM, e come i dati scorrono tra moduli, artefatti e decisioni. Sono inclusi diagrammi e codice estratto dai file principali del progetto per garantire la tracciabilità dei passaggi fondamentali.

Ambito: esecuzione da ``start.py``, generazione di artefatti, decisione VLM (prompt + immagini), rilevamento YOLO, esecuzione delle azioni nel simulatore, e produzione della documentazione (overlay + video).

File chiave (estratti): ``Architettura/app/start.py``, ``Architettura/ai2thor/init_controller.py``, ``Architettura/ai2thor/utils.py``, ``Architettura/ai2thor/context.py``, ``Architettura/vlm/client.py``, ``Architettura/vlm/runner.py``, ``Architettura/vlm/agent/action_executor.py``, ``Architettura/yolo/utils.py``, ``Tirocinio/src/YOLO_vs_VLM/yolo_bounding_box.py``.

1. Architettura generale

L'architettura e' orchestrata da `start.py`, che coordina: (1) inizializzazione del controller AI2-THOR, (2) acquisizione frame e metadati, (3) YOLO per localizzare il target, (4) costruzione del contesto per la VLM, (5) generazione del piano e azioni, (6) esecuzione azioni sul simulatore, (7) salvataggio artefatti e documentazione finale.

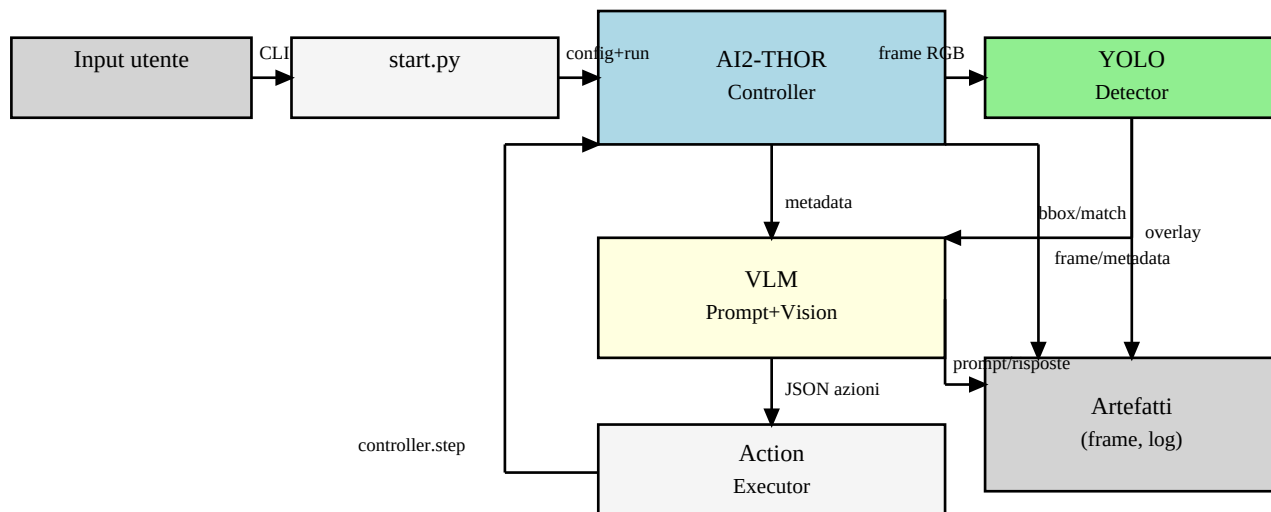


Figura 1 - Diagramma dei componenti principali (layout migliorato).

Nodi chiave: AI2-THOR produce immagini e metadata; YOLO produce bounding box e match del target; la VLM riceve immagini + contesto JSON e restituisce un JSON con azioni; Action Executor valida e invia comandi al controller. Ogni step genera artefatti per debug e documentazione (frame RGB/semantic/overlay, risposte VLM, log).

2. Avvio e configurazione (start.py + init_controller.py)

L'esecuzione inizia da `start.py` con il comando esempio: `VLMPROMPT=5 python3 start.py "cerca la bottiglia"`. Il parametro VLMPROMPT seleziona il prompt di sistema, mentre l'argomento testuale definisce il target.

`SimConfig` in `init\_controller.py` definisce scena, dimensioni, headless, grid e lo spawn controllato (Teleport iniziale). Il controller AI2-THOR viene creato con render di depth, instance e semantic.

Estratto codice (start.py):

```
# start.py (estratto)
config = SimConfig(scene="FloorPlan7")
controller = create_controller(config)
controller.step(
    action="AddThirdPartyCamera",
    position={"x": -1.25, "y": 1, "z": -1},
    rotation={"x": 90, "y": 0, "z": 0},
    fieldOfView=90,
)
```

Estratto codice (init_controller.py):

```
# init_controller.py (estratto)
controller = Controller(
    scene=config.scene,
    width=config.width,
    height=config.height,
    renderDepthImage=True,
    renderInstanceSegmentation=True,
    renderSemanticSegmentation=True,
    snapToGrid=False,
)
if config.agent_position or config.agent_rotation or config.agent_horizon:
    controller.step(
        action="Teleport",
        position=config.agent_position,
        rotation=config.agent_rotation,
        horizon=config.agent_horizon,
    )
```

Pseudocodice avvio:

PSEUDOCODICE AVVIO

1. Leggi prompt e input utente.
2. Crea SimConfig (scena, risoluzione, spawn controllato).
3. Avvia Controller AI2-THOR.
4. Aggiungi third-person camera.
5. Inizializza VLM con system prompt selezionato.

3. Ciclo runtime e acquisizione dei frame

Dopo l'inizializzazione, il sistema entra nel loop di steps. Ogni iterazione salva gli artefatti visivi (RGB, depth, semantic, third-person) e calcola informazioni YOLO. Questi dati alimentano il contesto VLM e la documentazione.

Estratto codice (save_artifacts):

```
# ai2thor/utils.py (estratto)
paths = save_artifacts(event, step_idx, target_label=current_target)
# salva RGB/depth/semantic + overlay YOLO
# ritorna yolo_target_match, target_center_x, ecc.
```

Nel loop si aggiorna anche la third-person camera per mantenere consistenza tra i frame esterni e quelli di overlay. Il risultato e' una sequenza di file in `Artefatti/vision\_outputs` e `Artefatti/yolo\_outputs`.

Pseudocodice ciclo principale:

```
PSEUDOCODICE STEP
for step in range(max_steps):
    if step == 0: event = controller.step("Pass")
    update_third_person_camera()
    paths = save_artifacts(event)
    estrai yolo_match, target_center_x, target_distance
    costruisci prompt e contesto
    run_vlm_on_step()
    execute_vlm_action()
    aggiorna scan_count e stato
```

4. Pipeline YOLO e traduzione target

Il target dell'utente viene estratto in italiano e tradotto in inglese per YOLO. `translate/translator.py` isola l'oggetto (rimuove articoli e stop-words) e, se disponibile, usa Argos Translate per ottenere la traduzione.

Estratto codice (translator):

```
# translate/translator.py (estratto)
user_input = "cerca la bottiglia"
target_it = extract_target_phrase(user_input)
translated_target = translate_target(user_input)
```

yolo_engine_17.py fa già tutta l'inferenza: carica il modello, predice, filtra per label, sceglie il bbox migliore e ritorna (bbox, debug_info), più un debug interno grezzo. yolo_bounding_box.py non aggiunge logica di detection: è un wrapper che gestisce input/format, disegna il bbox e salva l'overlay su disco, con una CLI comoda.

Estratto codice (yolo_engine_17.py):

```
# yolo_engine_17.py (estratto)
results = self.model.predict(source=inference_source, conf=conf, imgsz=640, verbose=False)
if not results: return None, "No results"
res0 = results[0]
boxes = res0.boxes
if boxes is None or len(boxes) == 0: return None, "No detections"

# Selezione
candidates = [d for d in detections if d["is_valid"]]
if candidates:
    candidates.sort(key=lambda d: d["score"], reverse=True)
    best = candidates[0]
    return best["bbox"], f"Found '{best['label']}' conf={best['score']:.2f}"
```

Pseudocodice YOLO:

PSEUDOCODICE YOLO

1. Leggi frame RGB dal simulatore.
2. Se target è vuoto, salva frame e termina.
3. Esegui YOLO (Ultralytics) con modello scelto.
4. Se target trovato, salva bbox/overlay.
5. Calcola centro X normalizzato per il prompt VLM.
6. Ritorna match=True/False + metadati.

5. Costruzione del contesto (AI2-THOR + YOLO)

`ai2thor/context.py` crea un contesto JSON per la VLM usando metadata, distanza dal target, collisioni e memoria dell'ultima posizione vista. Il modulo tenta di leggere un JSON YOLO (se presente) per calcolare last_seen; in caso contrario usa direttamente i metadata di AI2-THOR.

Estratto codice (build_context):

```
# context.py (estratto)
context = {
    "last_action": metadata.get("lastAction"),
    "last_action_success": metadata.get("lastActionSuccess", True),
    "target_distance": target_distance,
    "target_position": target_position,
    "collided": metadata.get("collided"),
}
```

Pseudocodice contesto:

PSEUDOCODICE CONTESTO

1. Leggi metadata AI2-THOR (oggetti, collisioni, ultima azione).
2. Se esiste yolo_json, calcola last_seen e posizione target nel frame.
3. Calcola target_distance dal metadata (istanza piu' vicina).
4. Costruisci dict con campi essenziali.
5. Ritorna JSON per il prompt VLM.

6. VLM: prompt, immagini e generazione

La VLM usa un system prompt selezionato con `VLMPROMPT` e un `user\_prompt` costruito in `start.py` (base_task). Il client VLM compone system+user+context e genera un JSON di azioni a partire da immagini (semantic + overlay YOLO).

Estratto codice (selezione prompt):

```
# vlm_prompt.py (estratto)
SYSTEM_PROMPTS = {"1": SYSTEM_PROMPT, "5": SYSTEM_PROMPT_FINAL, ...}

def select_system_prompt(name: str | None) -> str:
    key = (name or "").strip().lower()
    return SYSTEM_PROMPTS.get(key, SYSTEM_PROMPT)
```

Estratto codice (runner VLM):

```
# vlm/runner.py (estratto)
context = build_context(...)
prompt = vlm_client.compose_prompt(user_prompt, context=context)
images = [semantic_image] if disponibile else [rgb_frame]
if overlay_yolo: images.append(overlay_yolo)
answer = vlm_client.generate(image=images, prompt=prompt)
```

Pseudocodice VLM:

PSEUDOCODICE VLM

1. Costruisci contesto JSON dal simulatore.
2. Componi prompt = system_prompt + user_prompt + context.
3. Prepara immagini: semantic + overlay YOLO (se esistono).
4. Chiedi alla VLM di restituire JSON di azioni.
5. Salva prompt/risposta in Artefatti/vlm_outputs.

7. Parsing JSON ed esecuzione azioni

Il modulo `action\_executor.py` filtra le azioni consentite e gestisce JSON imperfetti (repair e fallback).
Se la VLM restituisce una sequenza di azioni, vengono eseguite in ordine fino a Stop o fallimento.

Estratto codice (esecuzione):

```
# action_executor.py (estratto)
payload = parse_vlm_json(vlm_output)
sequence = payload.get("action_sequence")
if sequence:
    for step in sequence:
        action, params = normalize_action(step)
        event = controller.step(action=action, **params)
        if action == "Stop" or not success:
            break
else:
    action, params = normalize_action(payload)
    event = controller.step(action=action, **params)
```

Pseudocodice Action Executor:

PSEUDOCODICE AZIONI

1. Parse JSON (anche se sporco o incompleto).
2. Verifica che l'azione sia nella whitelist.
3. Normalizza parametri (cm -> metri).
4. Esegui controller.step().
5. Se action_sequence: continua finche' success o Stop.
6. Ritorna event + log dell'esecuzione.

8. Documentazione (overlay + video)

A fine simulazione, il sistema puo' avviare la documentazione: genera overlay con azioni e crea video. Il flusso e' in `Documentazione\_e\_Test/run/start\_documentation.py`. Dopo il salvataggio, viene eseguita la pulizia degli artefatti temporanei.

Estratto codice (documentazione):

```
# start_documentation.py (estratto)
overlay_cmd = [python, generate_action_overlay.py, --sim-name SIM_x]
video_cmd = [python, generate_video.py, --sim-name SIM_x]
run(overlay_cmd); run(video_cmd)
clear_artifacts(ARTIFACTS_ROOT)
```

Pseudocodice documentazione:

PSEUDOCODICE DOCUMENTAZIONE

1. Determina cartella SIM_n.
2. Genera overlay per ogni frame YOLO.
3. Crea video con ffmpeg.
4. Salva tutto in Artefatti/Documentazione/SIM_n.
5. Pulisci file temporanei in Artefatti/.

9. Pseudocodice end-to-end e flusso dati

Questa sezione sintetizza il flusso completo, includendo l'avvio, il loop decisionale e l'integrazione tra AI2-THOR, YOLO e VLM.

Pseudocodice end-to-end:

```
PSEUDOCODICE COMPLETO
input = CLI("cerca la bottiglia")
config = SimConfig(scene, resolution, spawn)
controller = create_controller(config)
add_third_person_camera()

# prepara VLM
prompt_key = ENV["VLMPROMPT"]
system_prompt = select_system_prompt(prompt_key)
user_prompt = build_base_task(input)
vlm = create_vlm(model_name, system_prompt, user_prompt)

# loop
for step in range(max_steps):
    if step == 0: event = controller.step("Pass")
    update_third_person_camera(event)
    paths = save_artifacts(event, target_label)
    context = build_context(event, target_label, yolo_json_path=paths.yolo_json)
    context.yolo_target_match = paths.yolo_target_match
    prompt = vlm.compose_prompt(user_prompt, context)
    images = [semantic] or [rgb]; if overlay: images += [overlay]
    vlm_output = vlm.generate(images, prompt)
    result = execute_vlm_action(controller, vlm_output)
    if result.action == "Stop": break
    event = result.event

# fine
se Ctrl+C: chiedi se avviare documentazione
se YES: run_documentation()
controller.stop()
```

Diagramma testuale del flusso:

```
FLUSSO DATI (ASCII)
Utente -> start.py
        -> AI2-THOR (frame + metadata)
        -> YOLO (bbox + match)
        -> build_context (JSON)
        -> VLM (prompt + immagini)
        -> Action Executor (JSON -> azioni)
        -> AI2-THOR (nuovo stato)
        -> Artefatti + Documentazione
```